TP3 – Les Layouts Android (Android)

Développement Mobile – BTS

Objectif général

L'objectif de ce TP est de permettre à l'étudiant de maîtriser la conception d'interfaces graphiques sous Android en utilisant les différents types de layouts.

À l'issue de ce TP, l'étudiant sera capable de :

- comprendre le rôle des layouts dans une application Android
- organiser une interface graphique de manière structurée et lisible
- utiliser le `LinearLayout` avec ses différentes orientations
- imbriquer plusieurs layouts pour construire une interface complète
- appliquer les notions de largeur, hauteur, marges et espacements
- reproduire une interface graphique à partir d'une maquette (capture)
- analyser les limites d'un layout simple face à une interface réelle

Ce TP vise également à développer la capacité de l'étudiant à choisir le layout le plus adapté selon la structure de l'interface à réaliser.

Interface cible à réaliser

L'interface à réaliser correspond à un écran de connexion (*Login Screen*) destiné à une application mobile.

Cette interface comprend les éléments suivants :

- Une zone supérieure contenant :
  - un titre indiquant l'existence d'un compte utilisateur
- Une zone centrale contenant un formulaire :
  - un champ de saisie pour l'adresse email
  - un champ de saisie pour le mot de passe
  - un bouton permettant de se connecter
  - un texte proposant l'inscription pour les nouveaux utilisateurs
- Une zone inférieure proposant des méthodes de connexion alternatives :
  - un texte de séparation indiquant l'utilisation d'autres méthodes
  - deux icônes représentant des réseaux sociaux

L'interface doit être claire, bien espacée et structurée, en respectant la hiérarchie visuelle entre les différentes zones.

Ce travail permet de comparer les approches, d'identifier leurs limites et de comprendre pourquoi **ConstraintLayout est aujourd'hui le layout recommandé**.

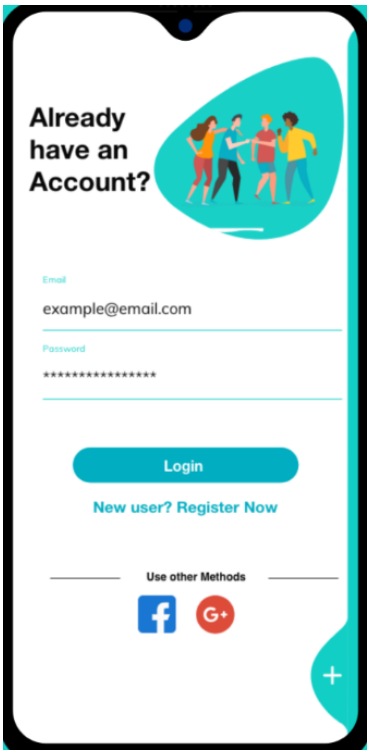

FIGURE 1 – Exemple d'interface de Login

Ressources utilisées

Image de fond

L'image de fond utilisée dans ce TP est fournie via le lien suivant :

— `https://drive.google.com/file/d/17u5rwqFw0IGxZELHJDoRlyS1_jTydOi9/view`

Cette image doit être placée dans le dossier :

`res/drawable`

Icônes

Les icônes (Facebook, Google) sont téléchargées depuis le site :

— `https://www.flaticon.com`

Après téléchargement, les icônes doivent être placées dans le dossier `res/drawable`. L'utilisation se fait à des fins pédagogiques.

Composants graphiques utilisés

MaterialButton

Dans ce TP, le bouton standard `<Button>` n'est pas utilisé. À la place, nous utilisons le composant suivant :

```
<com.google.android.material.button.MaterialButton>
```

Ce choix permet :

— une meilleure intégration avec le thème Material

— la gestion des couleurs via `backgroundTint`

— des coins arrondis avec `cornerRadius`
— un rendu moderne et professionnel

Utilisation du composant View comme séparateur

Le composant `View` est utilisé pour créer une ligne horizontale de séparation :

```
<View
    android:layout_width="0dp"
    android:layout_height="1dp"
    android:layout_weight="1"
    android:background="#CCCCCC" />
```

Ce composant est utilisé uniquement comme élément graphique.

Dans un premier temps, l'interface sera réalisée uniquement à l'aide du `LinearLayout`, avant d'être améliorée dans les parties suivantes du TP.

1 Partie 1 : Réalisation de l'interface avec `LinearLayout`

Dans cette première partie, l'interface de connexion est réalisée en utilisant le layout `LinearLayout`. L'objectif est de comprendre le principe d'organisation linéaire des composants et l'imbrication des layouts.

1.1 Étape 1 : Création du fichier layout

Créer un fichier nommé :

`activity_main_linear.xml`

Ce fichier doit être placé dans le dossier :

`res/layout`

Le layout racine est un `LinearLayout` avec une orientation verticale.

1.2 Étape 2 : Déclaration du LinearLayout racine

Le layout principal occupe tout l'écran et utilise une image de fond.

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@drawable/img4">
```

Remarque : L'orientation verticale permet d'empiler les composants de haut en bas.

1.3 Étape 3 : Ajout du titre

Un `TextView` est utilisé pour afficher le titre de l'interface.

```
<TextView

    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="20dp"
    android:layout_marginTop="130dp"
    android:text="Already have an account ?"
    android:textSize="30sp"
    android:textStyle="bold" />
```

À comprendre : Dans un `LinearLayout`, la position dépend uniquement de l'ordre des éléments dans le fichier XML.

1.4 Étape 4 : Champ Email

Le champ Email est composé de deux éléments :

- un `TextView` (libellé)
- un `EditText` (saisie)

```
<TextView
    android:text="Email"
    android:textColor="#34C3D5"
    android:textStyle="bold"
    android:layout_marginStart="30dp" />

<EditText
    android:layout_width="match_parent"
    android:layout_height="50dp"
    android:layout_marginStart="30dp"
    android:layout_marginEnd="30dp"
    android:hint="example@gmail.com"
    android:inputType="textEmailAddress" />
```

1.5 Étape 5 : Champ Password

Même principe que pour le champ Email.

```
<TextView
    android:text="Password"
    android:textColor="#34C3D5"
    android:textStyle="bold"
    android:layout_marginStart="30dp"
    android:layout_marginTop="10dp" />

<EditText
    android:layout_width="match_parent"
    android:layout_height="50dp"
    android:layout_marginStart="30dp"
    android:layout_marginEnd="30dp"
    android:hint="***************"
    android:inputType="textPassword" />
```

1.6 Étape 6 : Bouton de connexion

Le bouton utilisé est un `MaterialButton`, recommandé par Android.

```
<com.google.android.material.button.MaterialButton
    android:layout_width="192dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="30dp"
    android:layout_gravity="center_horizontal"
    android:text="Login"
    app:backgroundTint="@color/blue"
    app:cornerRadius="20dp" />
```

À comprendre : Le centrage est réalisé à l'aide de `layout_gravity` dans le conteneur linéaire.

1.7 Étape 7 : Texte d'inscription

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="80dp"
    android:text="New user? Register now"
    android:textAlignment="center"
    android:textStyle="bold" />
```

1.8 Étape 8 : Ligne de séparation

La ligne est réalisée à l'aide du composant `View`.

```
<View
    android:layout_width="0dp"
    android:layout_height="1dp"
    android:layout_weight="1"
    android:background="#CCCCCC" />
```

Cette ligne est combinée avec un texte dans un `LinearLayout` horizontal.

1.9 Étape 9 : Icônes de connexion

Les icônes sont placées dans un `LinearLayout` horizontal centré.

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center"
    android:orientation="horizontal">

    <ImageView
        android:layout_width="48dp"
        android:layout_height="48dp"
        android:src="@drawable/facebook" />

    <ImageView
```

```
        android:layout_width="48dp"
        android:layout_height="48dp"
        android:layout_marginStart="24dp"
        android:src="@drawable/google" />
</LinearLayout>
```

1.10 Bilan de la partie LinearLayout

Le `LinearLayout` permet une mise en page simple et lisible. Cependant, il nécessite :

— de nombreuses marges

— des imbrications supplémentaires

Il est adapté aux interfaces simples, mais devient limité pour des interfaces complexes.

2 Partie 2 : Réalisation de l'interface avec `RelativeLayout`

Dans cette partie, l'interface de connexion est réalisée à l'aide du `RelativeLayout`. Contrairement au `LinearLayout`, le positionnement des composants ne dépend plus uniquement de l'ordre dans le fichier XML, mais de **relations entre les vues**.

2.1 Notions fondamentales du RelativeLayout

Le `RelativeLayout` permet de positionner un composant :

— par rapport au **parent**

— par rapport à **un autre composant**

Pour cela, plusieurs attributs sont essentiels à comprendre.

Positionnement par rapport au parent

— `android:layout_alignParentTop="true"`

— `android:layout_alignParentBottom="true"`

— `android:layout_alignParentStart="true"`

— `android:layout_alignParentEnd="true"`

— `android:layout_centerHorizontal="true"`

Ces attributs permettent d'ancrer un composant par rapport aux bords de l'écran.

Positionnement par rapport à un autre composant

— `android:layout_below="@id/element"`

— `android:layout_above="@id/element"`

— `android:layout_toStartOf="@id/element"`

— `android:layout_toEndOf="@id/element"`

Remarque importante : Ces attributs nécessitent obligatoirement l'utilisation de l'attribut `android:id`.

Règles importantes à respecter

— Un composant ne doit pas dépendre de lui-même

— Il ne faut pas créer de dépendances circulaires

— Un seul type de positionnement par axe (vertical ou horizontal)

2.2 Étape 1 : Création du layout racine

Créer un fichier nommé :

`activity_main_relative.xml`

Le layout racine est un `RelativeLayout` occupant tout l'écran.

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/img4">
```

2.3 Étape 2 : Ajout du titre

Le titre est positionné par rapport au parent.

```
<TextView
    android:id="@+id/tvTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="33dp"
    android:layout_marginTop="92dp"
    android:text="Already have an account ?"
    android:textSize="30sp"
    android:textStyle="bold" />
```

2.4 Étape 3 : Champ Email

Le champ Email est positionné sous le titre.

```
<TextView
    android:id="@+id/tvEmail"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvTitle"
    android:layout_marginStart="63dp"
    android:layout_marginTop="24dp"
    android:text="Email"
    android:textColor="@color/blue" />

<EditText
    android:id="@+id/etEmail"
    android:layout_width="295dp"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvEmail"
    android:layout_marginStart="64dp"
    android:hint="aa@gmail.com"
    android:inputType="textEmailAddress" />
```

2.5 Étape 4 : Champ Password

Le champ Password est positionné sous le champ Email.

```
<TextView
    android:id="@+id/tvPassword"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/etEmail"
    android:layout_marginStart="63dp"
    android:layout_marginTop="16dp"
    android:text="Password"
    android:textColor="@color/blue" />

<EditText
    android:id="@+id/etPassword"
    android:layout_width="295dp"
    android:layout_height="wrap_content"
    android:layout_below="@id/tvPassword"
    android:layout_marginStart="64dp"
    android:hint="***************"
    android:inputType="textPassword" />
```

2.6 Étape 5 : Bouton de connexion

Le bouton est centré horizontalement par rapport au parent.

```
<com.google.android.material.button.MaterialButton
    android:id="@+id/btnLogin"
    android:layout_width="192dp"
    android:layout_height="wrap_content"
    android:layout_below="@id/etPassword"
    android:layout_marginTop="30dp"
    android:layout_centerHorizontal="true"
    android:text="Login"
    app:backgroundTint="@color/blue"
    app:cornerRadius="20dp" />
```

2.7 Étape 6 : Texte et séparation

Les éléments suivants sont positionnés successivement sous le bouton, en utilisant `layout_below`, et linear layout.

2.8 Bilan de la partie RelativeLayout

Le `RelativeLayout` offre plus de flexibilité que le `LinearLayout`, mais il nécessite :

— une gestion rigoureuse des identifiants

— une logique claire de dépendance entre composants

Une mauvaise organisation peut entraîner des erreurs de dépendances circulaires.

3 Partie 3 : Réalisation de l'interface avec `ConstraintLayout`

Dans cette partie, l'interface de connexion est réalisée à l'aide du `ConstraintLayout`. Ce layout est aujourd'hui **le plus recommandé par Android**, car il permet de créer des interfaces complexes avec **un seul layout racine** et sans imbrication excessive.

3.1 Notions fondamentales du ConstraintLayout

Le `ConstraintLayout` repose sur le principe de **contraintes**. Chaque composant doit être positionné :

- horizontalement (Start / End)
- verticalement (Top / Bottom)

Sans contraintes suffisantes, un composant est considéré comme **mal positionné**.

3.2 Contraintes par rapport au parent

Les contraintes suivantes permettent de positionner un composant par rapport à l'écran :

- `app:layout_constraintStart_toStartOf="parent"`
- `app:layout_constraintEnd_toEndOf="parent"`
- `app:layout_constraintTop_toTopOf="parent"`
- `app:layout_constraintBottom_toBottomOf="parent"`

Elles permettent d'ancrer un composant aux bords de l'écran.

3.3 Contraintes par rapport à un autre composant

Il est également possible de positionner un composant par rapport à un autre :

- `app:layout_constraintTop_toBottomOf="@id/element"`
- `app:layout_constraintBottom_toTopOf="@id/element"`
- `app:layout_constraintStart_toEndOf="@id/element"`
- `app:layout_constraintEnd_toStartOf="@id/element"`

Ces contraintes nécessitent obligatoirement l'utilisation d'identifiants (`id`).

3.4 Largeur adaptative : `0dp`

Dans un `ConstraintLayout`, une largeur définie à `0dp` signifie :

> La vue occupe tout l'espace disponible entre ses contraintes.

C'est une notion fondamentale pour créer des interfaces responsives.

3.5 Alignement vertical entre composants

Dans certains cas, on souhaite aligner verticalement plusieurs composants sur la même ligne horizontale.

Cela se fait à l'aide des contraintes suivantes :

- `app:layout_constraintTop_toTopOf="@id/element"`
- `app:layout_constraintBottom_toBottomOf="@id/element"`

Ces contraintes forcent les composants à partager la même position verticale.

3.6 Étude de cas : Ligne gauche – Texte – Ligne droite

Pour afficher une séparation du type :

```
---- Use other Methods ----
```

trois composants sont utilisés :

- une ligne gauche (`View`)
- un texte central (`TextView`)
- une ligne droite (`View`)

Pourquoi aligner les lignes sur le texte ? Les lignes gauche et droite doivent être alignées verticalement avec le texte central. C'est pourquoi les contraintes suivantes sont utilisées :

```
app:layout_constraintTop_toTopOf="@id/Other"
app:layout_constraintBottom_toBottomOf="@id/Other"
```

Explication :

- `Top_toTopOf` aligne le haut de la ligne avec le haut du texte
- `Bottom_toBottomOf` aligne le bas de la ligne avec le bas du texte

Ainsi, les lignes restent parfaitement centrées verticalement, quelle que soit la taille du texte ou de l'écran.

3.7 Contraintes horizontales des lignes

La ligne gauche utilise :

```
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toStartOf="@id/Other"
```

Elle s'étend donc entre le bord gauche de l'écran et le texte central.
La ligne droite utilise :

```
app:layout_constraintStart_toEndOf="@id/Other"
app:layout_constraintEnd_toEndOf="parent"
```

Elle s'étend entre le texte central et le bord droit de l'écran.

3.8 Rôle du biais horizontal

Le texte central est positionné avec :

```
app:layout_constraintHorizontal_bias="0.5"
```

Ce biais permet de centrer précisément le texte entre ses contraintes Start et End.

3.9 Règles importantes à retenir

- Chaque composant doit avoir au moins une contrainte horizontale
- Chaque composant doit avoir au moins une contrainte verticale
- Les marges servent à ajuster, pas à positionner
- `0dp` signifie largeur adaptative

3.10 Bilan de la partie ConstraintLayout

Le `ConstraintLayout` permet :

— une interface flexible et responsive

— une réduction de l'imbrication des layouts

— un contrôle précis du positionnement

Il constitue la solution recommandée pour les interfaces Android modernes.